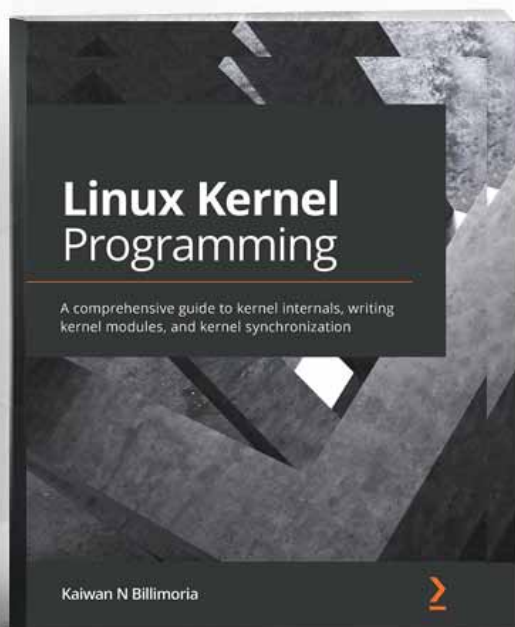


Linux Kernel Programming

‘Linux Kernel Programming’ is a book that is targeted at people who are quite new to the world of Linux kernel development, and makes no assumptions regarding their knowledge of the kernel. The only prerequisites to reading this book is knowing your way around Linux on the command line and a working knowledge of programming on Linux with ‘C’, as that is the medium that is used throughout it (along with a few bash scripts).

This comprehensive guide to kernel internals, writing kernel modules, and kernel synchronisation was published by Packt in March 2021. All the material and code examples in the book are based on the 5.4 LTS Linux kernel. This kernel is slated to be maintained right through December 2025, thus keeping the book’s content very relevant for a long while! It can be a handy resource for people dealing with device drivers, embedded Linux, the Linux kernel or Linux programming.

The book is divided into three major sections. The extract that follows is taken from Chapter 6 of the book, titled ‘Kernel Internals Essentials - Processes and Threads’, this chapter falls under the section ‘Understanding Process and Interrupt Contexts’.



In Chapter 4, *Writing Your First Kernel Module – LKMs, Part 1*, we presented a brief section entitled Kernel architecture I (if you haven’t read it yet, I suggest you do so before continuing). We will now expand on this discussion.

It’s critical to understand that most modern OSes are *monolithic* in design. The word monolithic literally means a single large piece of stone. We shall defer a little later to how exactly this applies to our favorite

OS! For now, we understand monolithic as meaning this: when a process or thread issues a system call, it switches to (privileged) kernel mode and executes kernel code, and possibly works on kernel data. Yes, there is no kernel or kernel thread executing code on its behalf; the process (or thread) itself executes kernel code. Thus, we say that kernel code executes within the context of a user space process or thread – we call this the process context. Think about it, significant portions of the kernel execute precisely this way, including a large portion of the code of device drivers.

Well, you may ask, now that you understand this, how else – besides process context – can kernel code execute? There is another way: when a hardware interrupt (from a peripheral device – the keyboard, a network card, a disk, and so on) fires, the CPU’s control unit saves the current context and immediately re-vectors the CPU to run the code of the interrupt handler (the interrupt service routine—ISR). Now this code runs in kernel (privileged) mode too – in effect, this is another, asynchronous, way to switch to kernel mode! The interrupt code paths of many device drivers are executed like this; we say that the kernel code being executed in this manner is executing in interrupt context.

So, any and every piece of kernel code is entered by and executes in one of two contexts: